Desarrollo orientado a objetos

23 de octubre de 2023

Material conceptual:

Acceso a Base de datos

Conocimiento previo requerido

Conceptos de Base de datos

Lenguaje de Manipulación de Datos (DML)

Lenguaje de Definición de Datos (DDL)

Temas incluidos

Base de datos SQLite

Conexión a base de datos

Cursores

Ejecutar operaciones con la base de datos

Uso de librería SQLite3

Motor de base de datos SQLite

No es necesario instalar SQLite por separado en tu PC cuando trabajas con Python, ya que SQLite es una parte integral de la biblioteca estándar de Python. En otras palabras, Python incluye el módulo sqlite3 para interactuar con bases de datos SQLite. Por lo tanto, no necesitas realizar una instalación adicional de SQLite en tu sistema.

Solo necesitas asegurarte de que Python esté correctamente instalado y que puedas importar el módulo sqlite3. Si puedes importarlo sin errores, entonces estás listo para empezar a trabajar con bases de datos SQLite desde Python. El ejemplo que te proporcioné en la respuesta anterior utiliza este módulo para interactuar con SQLite.

Librería para trabajar con base de datos SQLite

Para acceder a una base de datos SQLite desde Python, necesitas usar el módulo sqlite3, que es parte de la biblioteca estándar de Python. Aquí hay un ejemplo básico de cómo puedes crear una base de datos SQLite y realizar operaciones de lectura y escritura:

```python
import sqlite3

# Conectarse a la base de datos (asegúrate de tener un archivo 'mi_base_de_d
conn = sqlite3.connect('mi_base_de_datos.db')

# Crear un cursor
cursor = conn.cursor()

# Datos del registro que deseas insertar
nombre = 'Juan'
apellido = 'Pérez'
edad = 30

# Ejecutar una consulta SQL para insertar el registro en la tabla
cursor.execute("INSERT INTO mi_tabla (nombre, apellido, edad) VALUES (?, ?,

# Confirmar la transacción (guardar los cambios en la base de datos)
conn.commit()

# Cerrar el cursor y la conexión
cursor.close()
conn.close()
```

Lectura y escritura

```python
import sqlite3

# Conectar a la base de datos (o crearla si no existe)
conn = sqlite3.connect("mi_base_de_datos.db")

# Crear un cursor
cursor = conn.cursor()

# Crear una tabla
cursor.execute('''CREATE TABLE IF NOT EXISTS empleados
                  (id INTEGER PRIMARY KEY,
                   nombre TEXT,
                   salario REAL)''')

# Insertar datos en la tabla
cursor.execute("INSERT INTO empleados (nombre, salario) VALUES (?, ?)", ("Ju
cursor.execute("INSERT INTO empleados (nombre, salario) VALUES (?, ?)", ("Ma

# Guardar los cambios
conn.commit()
```

```python
# Consultar datos de la tabla
cursor.execute("SELECT * FROM empleados")
resultados = cursor.fetchall()
for fila in resultados:
    print(f"ID: {fila[0]}, Nombre: {fila[1]}, Salario: {fila[2]}")

# Cerrar la conexión a la base de datos
conn.close()
```

En este ejemplo:

Importamos el módulo sqlite3.

Conectamos o creamos una base de datos llamada "mi_base_de_datos.db".

Creamos un cursor que nos permite ejecutar comandos SQL en la base de datos.

Creamos una tabla llamada "empleados".

Insertamos datos en la tabla.

Consultamos datos de la tabla y los imprimimos.

Finalmente, cerramos la conexión a la base de datos.

Este es un ejemplo simple para empezar. Puedes adaptar y expandir el código según tus necesidades para trabajar con bases de datos SQLite desde Python. Asegúrate de gestionar las excepciones adecuadamente y seguir las mejores prácticas de seguridad, especialmente si estás trabajando con datos sensibles.

Cursores en Python

En Python, un cursor se refiere a un objeto proporcionado por módulos de bases de datos, como sqlite3 (para SQLite) o psycopg2 (para PostgreSQL), que permite ejecutar consultas SQL y recuperar datos de la base de datos. Un cursor actúa como una interfaz entre tu programa y la base de datos. Aquí hay una breve descripción de su uso:

Conexión a la Base de Datos: En primer lugar, debes establecer una conexión a la base de datos. Utilizas un objeto de conexión para conectarte a la base de datos utilizando información como el nombre del archivo de la base de datos, el nombre de usuario, la contraseña, etc.

Crear un Cursor: Una vez que tengas una conexión activa a la base de datos, creas un objeto cursor utilizando el método cursor() de la conexión. Por ejemplo:

```
import sqlite3
conn = sqlite3.connect('mi_base_de_datos.db')
cursor = conn.cursor()
```

Ejecutar Consultas SQL: Utilizas el cursor para ejecutar consultas SQL, como SELECT, INSERT, UPDATE o DELETE, utilizando métodos como `execute()`. Por ejemplo:

```
cursor.execute("SELECT * FROM mi_tabla")
```

Recuperar Datos: Utilizas el cursor para recuperar datos resultantes de la consulta, como filas de una tabla. Puedes iterar sobre las filas o utilizar métodos como fetchall(), fetchone(), etc., para recuperar datos.

Cerrar el Cursor y la Conexión: Cuando hayas terminado de usar el cursor, es una buena práctica cerrarlo y también cerrar la conexión a la base de datos para liberar recursos. Puedes hacerlo con los métodos close():

```
cursor.close()
conn.close()
```

Los cursores son una parte esencial cuando trabajas con bases de datos en Python, ya que te permiten interactuar con la base de datos de una manera segura y eficiente.

Métodos más usados en Cursores:

Con un cursor en SQLite, puedes ejecutar varios métodos para interactuar con la base de datos. Algunos de los métodos más comunes son:

1. **execute(query, parameters)**: Ejecuta una consulta SQL en la base de datos. Puedes pasar una cadena SQL como el parámetro **query**. Si la consulta SQL tiene parámetros, puedes proporcionarlos como una tupla o lista en el parámetro **parameters**.
2. **executemany(query, seq_of_parameters)**: Ejecuta una consulta SQL múltiples veces con diferentes conjuntos de parámetros. **query** es la consulta SQL, y **seq_of_parameters** es una secuencia que contiene varias tuplas (o listas) de parámetros.
3. **fetchone()**: Recupera la siguiente fila del resultado de una consulta como una tupla, o **None** si no hay más filas.
4. **fetchall()**: Recupera todas las filas restantes del resultado de una consulta como una lista de tuplas.
5. **fetchmany(size)**: Recupera varias filas (especificadas por **size**) del resultado de una consulta como una lista de tuplas.
6. **fetchall()**: Recupera todas las filas restantes del resultado de una consulta como una lista de tuplas.
7. **executeScript(script)**: Ejecuta múltiples sentencias SQL contenidas en una cadena. Esta función se utiliza comúnmente para ejecutar secuencias de instrucciones separadas por punto y coma.
8. **close()**: Cierra el cursor, liberando recursos. Después de cerrar un cursor, no se pueden ejecutar más consultas.

Estos son algunos de los métodos más utilizados. El uso de estos métodos depende de tus necesidades específicas al interactuar con la base de datos SQLite.

Ejemplo de recorrido de una tabla

```
import sqlite3

# Conectarse a la base de datos (o crearla si no existe)
conn = sqlite3.connect("mi_base_de_datos.db")

# Crear un cursor
cursor = conn.cursor()

# Ejecutar una consulta
cursor.execute("SELECT * FROM mi_tabla")

# Recorrer las filas del resultado
row = cursor.fetchone()
while row is not None:
    # Procesar la fila, por ejemplo, imprimir todos los campos
    print("Nombre:", row[0])
    print("Edad:", row[1])
    print("Dirección:", row[2])

    # Obtener la siguiente fila
    row = cursor.fetchone()

# Cerrar el cursor y la conexión
cursor.close()
conn.close()
```

Este código:

1. Abre una conexión con una base de datos SQLite llamada "mi_base_de_datos.db" (o la crea si no existe).
2. Crea un cursor para ejecutar consultas en la base de datos.
3. Ejecuta una consulta SQL para seleccionar todos los registros de una tabla llamada "mi_tabla".
4. Utiliza un bucle **while** para recorrer las filas del resultado. Con **fetchone()**, obtiene una fila a la vez y la almacena en la variable **row**.

5. Dentro del bucle, procesa y muestra los datos de cada fila (en este caso, el nombre, edad y dirección).
6. Luego, obtiene la siguiente fila con **fetchone()** y continúa el bucle hasta que no haya más filas.
7. Finalmente, cierra el cursor y la conexión a la base de datos.

Recorrer N registros en una tabla

En este ejemplo, supongamos que tienes una tabla llamada "mi_tabla" en una base de datos SQLite y deseas recuperar los primeros N registros de esa tabla:

```
import sqlite3

# Conectarse a la base de datos (asegúrate de tener un archivo 'mi_base_de_
conn = sqlite3.connect('mi_base_de_datos.db')

# Crear un cursor
cursor = conn.cursor()

# Definir el número de registros a recuperar (por ejemplo, los primeros 10 
N = 10

# Ejecutar una consulta SQL para recuperar los primeros N registros
cursor.execute("SELECT * FROM mi_tabla LIMIT ?", (N,))

# Recorrer y mostrar los resultados
for row in cursor.fetchall():
    print(row)  # Esto mostrará cada fila como una tupla

# Cerrar el cursor y la conexión
cursor.close()
conn.close()
```

Para mostrar una sola columna

```
import sqlite3

# Conectarse a la base de datos (asegúrate de tener un archivo 'mi_base_de_d
conn = sqlite3.connect('mi_base_de_datos.db')

# Crear un cursor
cursor = conn.cursor()

# Definir el número de registros a recuperar (por ejemplo, los primeros 10 r
N = 10

# Ejecutar una consulta SQL para recuperar los primeros N registros y sus ap
cursor.execute("SELECT apellido FROM mi_tabla LIMIT ?", (N,))

# Recorrer y mostrar los apellidos
for row in cursor.fetchall():
    print(row[0])  # Esto mostrará solo el apellido de cada fila

# Cerrar el cursor y la conexión
cursor.close()
conn.close()
```

Para agregar registros en una base de datos.

```python
import sqlite3

# Conectarse a la base de datos (asegúrate de tener un archivo 'mi_base_de_d
conn = sqlite3.connect('mi_base_de_datos.db')

# Crear un cursor
cursor = conn.cursor()

# Datos del registro que deseas insertar
nombre = 'Juan'
apellido = 'Pérez'
edad = 30

# Ejecutar una consulta SQL para insertar el registro en la tabla
cursor.execute("INSERT INTO mi_tabla (nombre, apellido, edad) VALUES (?, ?,

# Confirmar la transacción (guardar los cambios en la base de datos)
conn.commit()

# Cerrar el cursor y la conexión
cursor.close()
conn.close()
```

En este ejemplo, hemos creado una conexión a la base de datos, un cursor y luego hemos ejecutado una consulta SQL de inserción con la declaración INSERT INTO. Los valores se pasan como parámetros en una tupla (nombre, apellido, edad) que coincide con los valores correspondientes en la consulta.

Luego, para confirmar la inserción, se llama a commit() en la conexión, lo que guarda los cambios en la base de datos.

Manejo de errores

Es muy importante considerar que cuando realizamos operaciones en base datos pueden ocurrir errores y debemos manejarlos para evitar efectos no esperados e inconsistencias.

```python
import sqlite3

try:
    # Conecta a la base de datos (creará un archivo de base de datos si no
    connection = sqlite3.connect("mi_base_de_datos.db")

    # Crea un cursor para ejecutar comandos SQL
    cursor = connection.cursor()

    # Intenta insertar un nuevo registro
    cursor.execute("INSERT INTO tabla_ejemplo (campo1, campo2) VALUES (?, ?

    # Confirma los cambios en la base de datos
    connection.commit()

except sqlite3.Error as error:
    print("Error al insertar datos:", error)

finally:
    # Cierra la conexión a la base de datos
    connection.close()
```

En este ejemplo:

Importamos el módulo sqlite3 para trabajar con bases de datos SQLite.

Usamos un bloque try para intentar insertar datos en la base de datos.

Si ocurre algún error durante la inserción (por ejemplo, un problema de integridad o un campo duplicado), se captura la excepción sqlite3.Error y se imprime un mensaje de error.

Finalmente, cerramos la conexión a la base de datos en el bloque finally para asegurarnos de que los recursos se liberen adecuadamente.

Recuerda que debes reemplazar "mi_base_de_datos.db" por el nombre de tu propia base de datos y "tabla_ejemplo" por el nombre de tu propia tabla, además de ajustar los nombres de campo y valores según tus necesidades. También puedes personalizar el manejo de errores según tus requerimientos.

Uso de Rollback

```python
# Conectar a la base de datos SQLite
conn = sqlite3.connect('mi_base_de_datos.db')
cursor = conn.cursor()

try:
    # Iniciar una transacción
    conn.execute("BEGIN TRANSACTION;")

    # Realizar algunas operaciones
    cursor.execute("INSERT INTO empleados (nombre, salario) VALUES ('Carlos', 50000);")
    cursor.execute("INSERT INTO empleados (nombre, salario) VALUES ('Ana', 45000);")

    # Si todas las operaciones son exitosas, se hace commit
    conn.commit()
    print("Transacción completada con éxito.")

except Exception as e:
    # Si ocurre un error, hacer rollback para deshacer los cambios
    conn.rollback()
    print(f"Error durante la transacción: {e}")

finally:
    # Cerrar la conexión
    conn.close()
```

Ejercicios de practica

1. **Lectura de Datos**:
    - Escribe un programa que se conecte a una base de datos SQLite y recupere todos los registros de una tabla específica. Luego, muestra los registros en la consola.

- Crea una tabla que almacene información sobre libros (título, autor, año de publicación). Escribe un programa que lea y muestre todos los libros almacenados en la base de datos.

2. **Inserción de Datos**:
    - Crea una base de datos SQLite y una tabla que almacene información de estudiantes (nombre, edad, calificaciones). Escribe un programa que permita al usuario ingresar datos de nuevos estudiantes y los inserte en la base de datos.
    - Crea una base de datos que almacene información de productos (nombre, precio, cantidad en stock). Escribe un programa que permita al usuario ingresar nuevos productos y sus detalles en la base de datos.
3. **Actualización de Datos**:
    - Crea una base de datos que almacene información sobre empleados (nombre, salario, departamento). Escribe un programa que permita actualizar el salario de un empleado específico.
    - Crea una tabla que almacene información de contactos (nombre, correo electrónico, número de teléfono). Escribe un programa que permita al usuario actualizar el correo electrónico de un contacto.
4. **Eliminación de Datos**:
    - Crea una base de datos que almacene información de tareas pendientes (nombre de la tarea, fecha de vencimiento). Escribe un programa que permita al usuario eliminar una tarea específica de la base de datos.
    - Crea una tabla que almacene información de productos (nombre, precio, cantidad en stock). Escribe un programa que permita al usuario eliminar un producto de la base de datos.
5. **Consulta Avanzada**:
    - Escribe un programa que conecte a una base de datos de películas y muestre todas las películas de un género específico.
    - Crea una tabla que almacene información de clientes (nombre, dirección, fecha de registro). Escribe un programa que muestre todos los clientes que se registraron en un mes y año específicos.
6. **Exportación e Importación**:
    - Crea una base de datos que almacene información de contactos (nombre, correo electrónico, número de teléfono). Escribe un programa que exporte los datos de la base de datos a un archivo CSV.
    - Escribe un programa que importe datos de un archivo CSV a una tabla de la base de datos SQLite.

Material adicional:

Tutorial

https://likegeeks.com/python-sqlite3-tutorial/

Material de BD:

EjerciciosSQL.Pdf

ManualPracticoSQL.Pdf

AprendiendoSQL.Docx